02203 Design of Digital Systems (Fall 2026)

Assignment 2: Edge Detection

Advanced Didactic-SoC Version

An advanced SystemVerilog digital circuit design assignment using the Didactic-SoC, XILINX Vivado, and an Artix-7 FPGA board.

2026 v.8.1.SoC.LP

Abstract

This document describes the **advanced version** of the edge detection design project, which is part of course "02203 - Design of Digital Systems" at DTU. The project will result in an FPGA realization of an edge detection hardware accelerator for monochrome images. Edge detection is an important aspect of analyzing a video scene and can be used, for example, in surveillance systems. The edge detector is expected to identify abrupt intensity changes, which occur at the boundaries of objects. Further analysis of the edges may identify the type of object, e.g., a car or a human. In this advanced version, the accelerator is implemented in SystemVerilog and integrated into the Didactic-SoC, where a RISC-V CPU transports image data between a UART peripheral and the accelerator.

Important: this is the optional advanced version

This version of Assignment 2 is intended for students who would like an additional challenge and would like to work with an accelerator integrated into a complete system-on-chip.

Doing this version is **optional**; you are completely free to use the **baseline Assignment 2** instead. If you start the advanced version and later find that the SoC integration is taking too much time, you may switch back to the baseline assignment at any point.

The edge-detection problem is essentially the same in both versions. Although the surrounding interfaces are different, the Sobel algorithm, architecture and datapath design, ASMD chart, buffering ideas, and much of the accelerator design work remain useful if you switch. It is therefore **strongly recommended that you also take a look at the baseline Assignment 2**, to get an overview of the original accelerator architecture.

Contents

1 Introduction **2**

2 Edge detection **2**

3 The accelerator in the Didactic-SoC **5**

4 Design and implementation **7**

5 Your report **8**

1 Introduction

In this project, you will design an edge detection hardware (HW) accelerator for monochrome images. The images are represented in PGM format, and the size of an image frame is 352x288 pixels, i.e., 288 lines, each with 352 pixels. The intensity of a pixel is represented by an 8-bit number in the integer range from 0 to 255, where 0 and 255 refer to black and white, respectively. The objective is to implement a hardware accelerator that applies the Sobel edge-detection operation described in Section 2 to the image.

In the baseline assignment, the accelerator is used with a simpler, direct memory-oriented infrastructure. In this advanced version, the accelerator is instead integrated into a complete System-on-Chip (SoC), named **Didactic-SoC**. A RISC-V CPU receives image data through the UART peripheral and writes the pixels to an input buffer in student subsystem 0 (SS0). The accelerator processes the image and writes the result to an output buffer. The CPU then reads the processed image and sends it back to the PC over UART. The input and output buffers use 32-bit words, with four 8-bit pixels packed into each word.

Through the Didactic-SoC repository, you are provided with the SoC infrastructure, CPU firmware, testbenches, FPGA flow, and a working pixel-inversion accelerator. Your **main hardware-design task** is to replace the pixel-inversion processing module with your edge detector, verify the accelerator independently, and then verify it as part of the complete SoC and on the FPGA board.

The files and detailed instructions for the advanced assignment are provided through the following GitHub links:

- **Didactic-SoC repository (+ main README)**: This is the main repository. It is the entry point of the assignment. It contains the SystemVerilog source code, CPU firmware, testbenches, FPGA flow, working example accelerator, and supporting files used for the assignment. **Please look at the README file.**
- **Lab Guide**: Contains the official detailed assignment tasks, including the files to inspect or modify, commands to run, test procedures, and the purpose and success criteria for the individual tasks.
- **Software Setup Guide**: Explains how to install and verify the software tools required for the advanced assignment.

The **Lab Guide is the official detailed description of the assignment flow** and must be followed together with this document. This PDF explains the edge-detection problem, the main architecture, and the overall expectations, while the GitHub documentation contains the maintained implementation and tool instructions.

2 Edge detection

The algorithm used for *edge detection* is referred to as the Sobel operator. Edge detection is accomplished by computing the derivatives of the intensity signal in the x- and

y-directions. Thereafter, the minima and maxima in the derivatives need to be found, indicating where the intensity changes most rapidly.

The Sobel method approximates the derivatives in each direction by a process called *convolution*. In other words, convolution is performed by adding the weighted (filtered) intensities of the current pixel **and** the eight surrounding neighboring pixels. The coefficients of the weighting function are represented by convolution masks G_x and G_y for the derivatives in the x- and y-directions, respectively, as shown in Figure 1.

G_x=

-1	0	+1
-2	0	+2
-1	0	+1

G_y=

+1	+2	+1
0	0	0
-1	-2	-1

Figure 1: Sobel convolution masks.

The derivative image is computed by centering the convolution masks on each pixel. This needs to be done over successive pixels in the original frame. The coefficients in each mask are multiplied by the underlying pixel intensities, and the nine products (six of which are nonzero) are added to form two partial derivative values, D_x and D_y, as

$$D_x(n) = G_x * S(n) \tag{1}$$

and

$$D_y(n) = G_y * S(n) \tag{2}$$

where * and n denote the convolution operator and the pixel number in a scan line, respectively. The magnitude is approximated as

$$|D(n)| = |D_x(n)| + |D_y(n)| \tag{3}$$

Finally, the value computed in (3) is adapted to the numerical range used and stored in the accelerator output buffer, and the edge detector may continue with the next pixel.

The details of the Sobel calculation are illustrated below. Figure 2 shows the pixels in a 3×3 pixel fragment of a picture.

s_{11}	s_{12}	s_{13}
s_{21}	s_{22}	s_{23}
s_{31}	s_{32}	s_{33}

Figure 2: Matrix showing nine pixels.

The derivatives in the x and y directions for pixel s_{22} in the result picture are approximated by equations 4 and 5. In equation 4, the difference (or change) between the third and the first columns is calculated, and the pixels in the middle of the columns are weighted more strongly. The same applies to the first and the third rows in equation 5. The value calculated in equation 6 is adapted to the numerical range used and stored in the output image corresponding to pixel s_{22}. This value is large for large intensity changes and small for small intensity changes.

$$D_x(n) = s_{13} - s_{11} + 2(s_{23} - s_{21}) + s_{33} - s_{31} \tag{4}$$

$$D_y(n) = s_{11} - s_{31} + 2(s_{12} - s_{32}) + s_{13} - s_{33} \tag{5}$$

$$|D(n)| = |D_x(n)| + |D_y(n)| \tag{6}$$

2.1 About boundary conditions

The Sobel operation requires a 3×3 neighborhood around the pixel being processed. For pixels in the first or last row, or in the first or last column, part of this neighborhood lies outside the image and the corresponding pixel values therefore do not exist.

For the basic implementation, you may ignore these boundary conditions. In this case, the Sobel operator is applied only where the complete 3×3 neighborhood is available. The resulting image therefore does not contain the upper and lower rows and the left and right columns of the original image.

An optional improvement, included in Task 6 of the Lab Guide, is to also compute the boundary pixels. One simple technique is to conceptually extend the image by copying the values at its boundaries, as illustrated in Figure 3. The pixels shown outside the image boundary obtain their values from the corresponding pixels at the edge of the original image.

For example, to compute the Sobel value of the upper-left pixel $(0,0)$, values are required above it, to its left, and diagonally above-left. These positions can be filled by copies of the boundary pixels. In particular, pixel $(0,0)$ is used also for the three positions immediately above, to the left, and diagonally above-left. The same principle is applied to the remaining image edges and corners.

With this extension, a complete 3×3 neighborhood can be formed for every pixel, allowing the processed image to have the same dimensions as the input image. Handling the boundary conditions in this way is optional and is suggested as one of the improvements in Task 6.

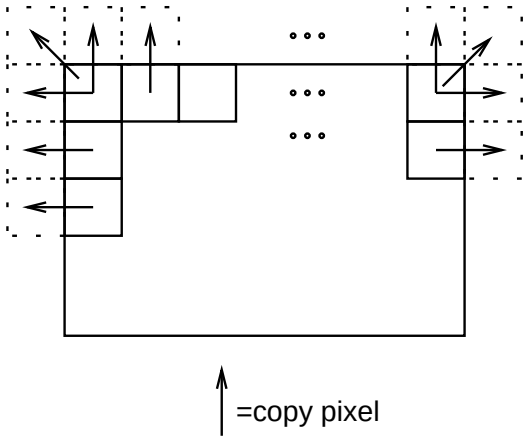


Figure 3: Boundary conditions. Pixels outside the image are obtained by copying values from the image boundary.

3 The accelerator in the Didactic-SoC

The advanced assignment uses the Didactic-SoC instead of the simpler direct-memory infrastructure used in the baseline assignment. The actual edge-detection problem is the same, but the accelerator now operates as part of a complete system containing a processor, memories, peripherals, and a bus.

The Didactic-SoC contains a RISC-V **Ibex** CPU, instruction and data memories, staff peripherals (UART, SPI and GPIO), and five student subsystem slots (SS0–SS4). The student subsystems are connected to the CPU through an **APB** bus and are memory-mapped into the CPU address space. Your image processing accelerator is located in **student subsystem 0 (SS0)**.

Figure 4 shows the organization of the Didactic-SoC. For this assignment, the important path is from the UART (in "Staff Peripherals") to the CPU and from the CPU through the APB interconnect to SS0. An image is sent from the PC over UART. The CPU receives the image and transfers it to the input buffer in SS0. The accelerator processes the image and stores the result in the output buffer. Finally, the CPU reads the processed image from the output buffer and sends it back to the PC over UART.

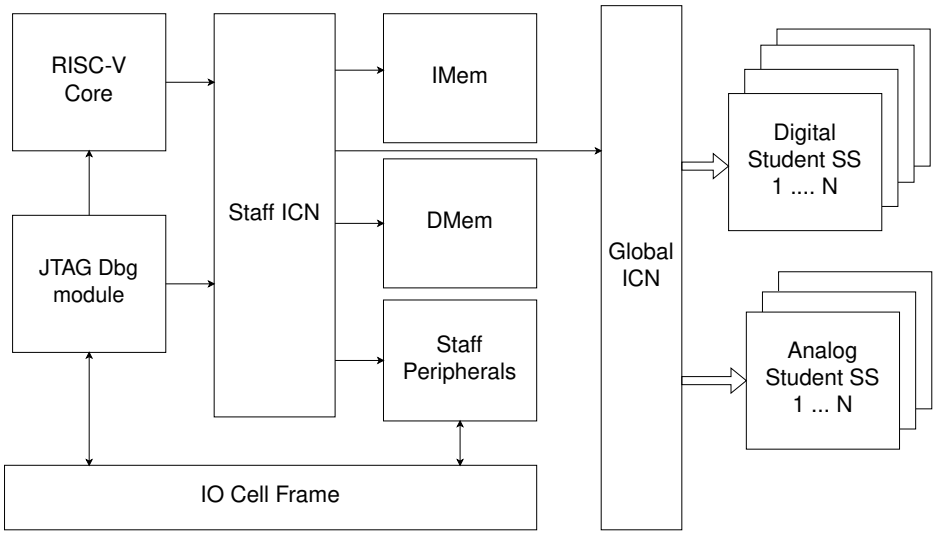


Figure 4: The Didactic-SoC architecture. The image accelerator used in this assignment is implemented in student subsystem 0 (SS0).

3.1 Student subsystem 0

The top-level student subsystem is provided in `Student_area_0.sv`. It contains the APB bus handling, a control/status register (CSR), an input buffer, an output buffer, and the pixel-processing submodule `pixel_acc.sv`. The supplied version is a complete working example that performs pixel inversion.

Figure 5 shows the organization of student subsystem 0. The CPU accesses the subsystem through the APB interface. The input and output buffers hold the image data, while the CSR provides the control and status interface between the CPU and the accelerator. The actual pixel-processing hardware is implemented in `pixel_acc.sv`.

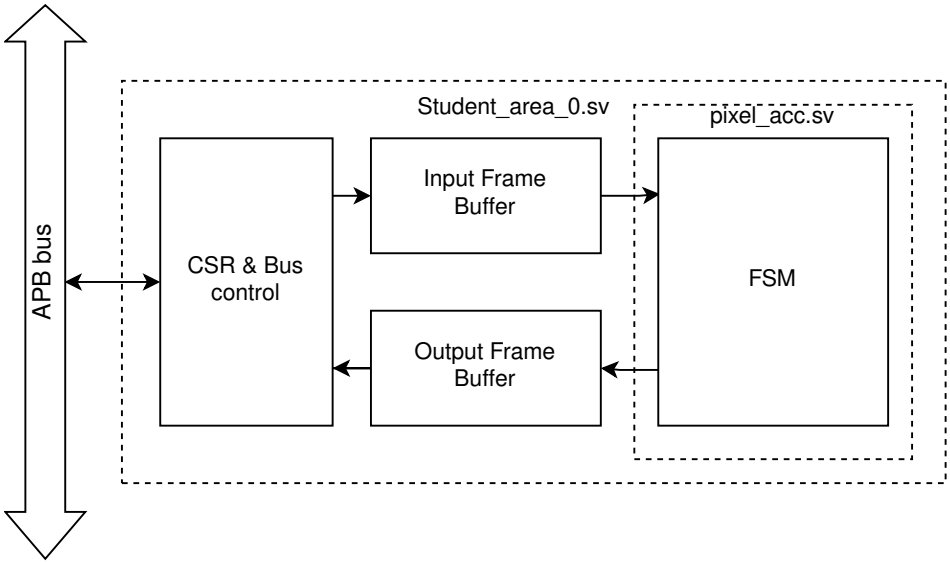


Figure 5: Organization of student subsystem 0 (SS0), including the APB interface, control/status register, input and output buffers, and the pixel-processing accelerator.

For the assignment, your edge detector is implemented primarily in `pixel_acc.sv`. The CPU/APB/CSR wrapper and the input/output-buffer infrastructure in `Student_area_0.sv` are provided and normally do not have to be redesigned. You should only need to modify the wrapper if you deliberately change the buffer organization or module parameters.

3.2 Image and buffer organization

The input image is a 352×288 monochrome image containing 8-bit pixels (288 rows of 352 pixels each). Pixel coordinates are written as (row, column), so $(0,0)$ is the upper-left pixel and $(287,351)$ is the lower-right pixel. Pixels are stored row by row: $(0,0)$, $(0,1)$, $\ldots$, $(0,351)$, followed by $(1,0)$, $(1,1)$, and so on.

The accelerator input and output buffers are organized as 32-bit words. Consequently, each buffer word contains four consecutive 8-bit pixels. The pixels are packed in little-endian order. Conceptually, the beginning of the input buffer therefore contains:

Buffer word	**bits 31–24**	**bits 23–16**	**bits 15–8**	**bits 7–0**
0	pixel $(0,3)$	pixel $(0,2)$	pixel $(0,1)$	pixel $(0,0)$
1	pixel $(0,7)$	pixel $(0,6)$	pixel $(0,5)$	pixel $(0,4)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Thus, one buffer address corresponds to four image pixels rather than to one pixel. The same packing is used for the output buffer.

Unlike the baseline assignment, the source and processed images are not stored in two regions of one common image RAM. SS0 contains a separate **input buffer** (*ibuf*) and **output buffer** (*obuf*). The CPU writes the received image into the input buffer, the accelerator reads the pixels from this buffer and writes the processed pixels to the output buffer, and the CPU subsequently reads the output buffer.

When designing the accelerator, remember that the input-buffer interface is synchronous: a read requested using `ibuf_rd_en` and `ibuf_rd_addr` produces the corresponding `ibuf_rd_data` with the one-cycle latency specified by the interface. The exact interface timing is documented in `pixel_acc.sv` and `Student_area_0.sv`.

3.3 Control and data flow

Conceptually, the CPU and accelerator cooperate as follows:

A) The CPU receives image data over UART and writes it to the accelerator input buffer.

B) The CPU indicates that the input data is ready by setting `DATA_READY` in the control/status register.

C) The wrapper starts `pixel_acc.sv`. The accelerator reads the input buffer, processes the image, and writes the result to the output buffer.

D) When processing is complete, the accelerator generates its `finish` indication and the wrapper exposes completion to the CPU through `DONE`.

E) The CPU reads the output buffer and sends the processed image back to the PC over UART.

The CPU is responsible for moving the image data between the UART and the accelerator buffers; the accelerator itself performs the image processing. The exact signal timing, buffer layout, and interface behavior are documented in the header comments of `pixel_acc.sv` and `Student_area_0.sv`, and in the GitHub Lab Guide. These sources provide the official description of the implementation interface.

4 Design and implementation

The advanced assignment is **SystemVerilog-based**.

This section provides only a **generic overview** of the design and implementation flow. It is not a replacement for the detailed instructions in the GitHub documentation. You must follow the official Lab Guide for the individual tasks, commands, files, paths, test procedures, and success criteria, and the Software Setup Guide for installation and verification of the required tools.

For convenience, the three main resources links are repeated here:

- **Didactic-SoC repository (+ main README)** – SystemVerilog source code, CPU firmware, testbenches, example accelerator, supporting files, and FPGA flow.
- **Lab Guide** – the official detailed description of the tasks, files, commands, test procedures, and success criteria.
- **Software Setup Guide** – installation and verification of the software tools required for the assignment.

The Lab Guide contains seven numbered tasks, Tasks 0–6. Tasks 0–4 form the required assignment flow; Tasks 5 and 6 are optional. For Tasks 0–5, the guide explicitly states the **purpose** of the task and the **success criteria**, so that it is clear what the task is intended to exercise and when it has been completed. Task 6 is an open-ended set of optional improvements.

The following list is only a summary of the progression. Refer to the **Lab Guide for the complete instructions for every task**:

Task 0: Test the Didactic-SoC with a Working Example. Verify the provided pixel-inversion example and the tool flow before changing the RTL.

Task 1: Develop the Edge Detection Accelerator. Design the accelerator and its datapath/ASMD chart, implement it in `pixel_acc.sv`, and verify it independently.

Task 2: Integrate the Accelerator into the SoC. Verify the accelerator when it is driven by the RISC-V CPU through the complete Didactic-SoC.

Task 3: FPGA Implementation. Synthesize and implement the complete design, generate a bitstream, and program the FPGA board.

Task 4: Test with the Serial Interface GUI. Perform the complete PC-to-SoC-to-accelerator-to-PC image round trip over UART.

Task 5 (Optional): Program and Debug over JTAG. Use OpenOCD and GDB for optional low-level software/debug work without rebuilding the bitstream for every firmware change.

Task 6 (Optional): Further improvements. Explore enhancements such as the boundary handling described in Section 2.1, pipelined transfers, or direct memory access (DMA).

Again, this summary is intended only to show the overall progression of the assignment. The official Lab Guide must be followed when carrying out the tasks.

5 Your report

Write a short report describing your design and your solutions to the required tasks in the Lab Guide. The report should explain the architecture of your edge detector, including the datapath and ASMD chart, and discuss the RTL implementation, standalone verification, SoC integration, and FPGA results as relevant. If you carry out optional work, you may also describe the corresponding results.

Each team must hand in an electronic version of the report in PDF format and the source files as a ZIP archive. In the ZIP file, include the SystemVerilog source files you have written or modified and any other relevant files you have produced. *Do not include large generated tool/project files.* The hand-in is completed by one student. The remaining students in the group should make sure they join the group so that we have everybody registered. The deadline is stated in the assignment facility of DTU-Learn.

The report must have a cover page containing the following elements: (i) the title of the report, (ii) your team number, and (iii) the name and student ID of each team member. In addition, you must include the following statement at the bottom of the cover page:

We confirm that this report contains our own independent and original work. Unless otherwise explained in the preface of the report, all group members have contributed equally to all parts of the work.

The report (and this Assignment) will contribute approximately 35% towards your final grade in this course.

Acknowledgment: The development of this assignment was funded by the European Union through the European Health and Digital Executive Agency (HaDEA) under Grant Agreement No. 101123086 (Edu4Chip). Views and opinions expressed are, however, those of the authors only and do not necessarily reflect those of the European Union or HaDEA. Neither the European Union nor the granting authority can be held responsible for them.